

Demonstration: Quarto, graphs, tables, and websites

Rohan Alexander

16 September, 2026

What Quarto is

Quarto turns a plain-text file with a `.qmd` extension into a PDF, a Word document, a slide deck, or a website. The file has three kinds of content: a YAML header between `---` lines that says what to produce; Markdown prose; and code chunks that run when the file renders, so the numbers, graphs, and tables in the output come from the data every time. This document is a `.qmd`; open `02-demo.qmd` next to the PDF and compare.

A paper where the figures were pasted in by hand cannot be checked and cannot be updated when the data change. A Quarto paper can be rendered by anyone with the repo.

The workflow from last week

Step 1: Python makes the data

`aggregate_shelter.py` downloads the City of Toronto's daily shelter occupancy file (the Week 1 data), keeps the columns we need, and aggregates to one row per date and sector. Run it once:

```
uv run aggregate_shelter.py
```

It writes `data/shelter_daily.parquet`. Parquet rather than CSV!

Step 2: R reads it

This chunk runs when this document renders. The first one reads the Parquet file with the **arrow** package.

```
library(arrow)
library(dplyr)
library(ggplot2)
library(tinytable)

daily <- read_parquet("data/shelter_daily.parquet")

glimpse(daily)
```

```
Rows: 1,285
Columns: 7
$ date          <date> 2026-01-01, 2026-01-01, 2026-01-01, 2026-01-01, 2026-0~
$ sector        <chr> "Families", "Men", "Mixed Adult", "Women", "Youth", "Fa~
$ service_users <int> 2320, 2024, 2974, 1084, 741, 2298, 2026, 2975, 1080, 74~
$ occupied_beds <int> 10, 1680, 1536, 834, 741, 10, 1682, 1537, 829, 740, 10,~
$ capacity_beds <int> 21, 1732, 1622, 866, 760, 21, 1732, 1607, 859, 756, 21,~
$ occupied_rooms <int> 705, 297, 1110, 193, 0, 699, 297, 1111, 194, 0, 700, 29~
$ capacity_rooms <int> 729, 297, 1117, 193, 0, 727, 298, 1118, 195, 0, 728, 29~
```

Step 3: a graph

```
daily |>
  ggplot(aes(x = date, y = service_users, colour = sector)) +
  geom_line() +
  labs(x = "Date", y = "People", colour = "Sector") +
  theme_minimal()
```

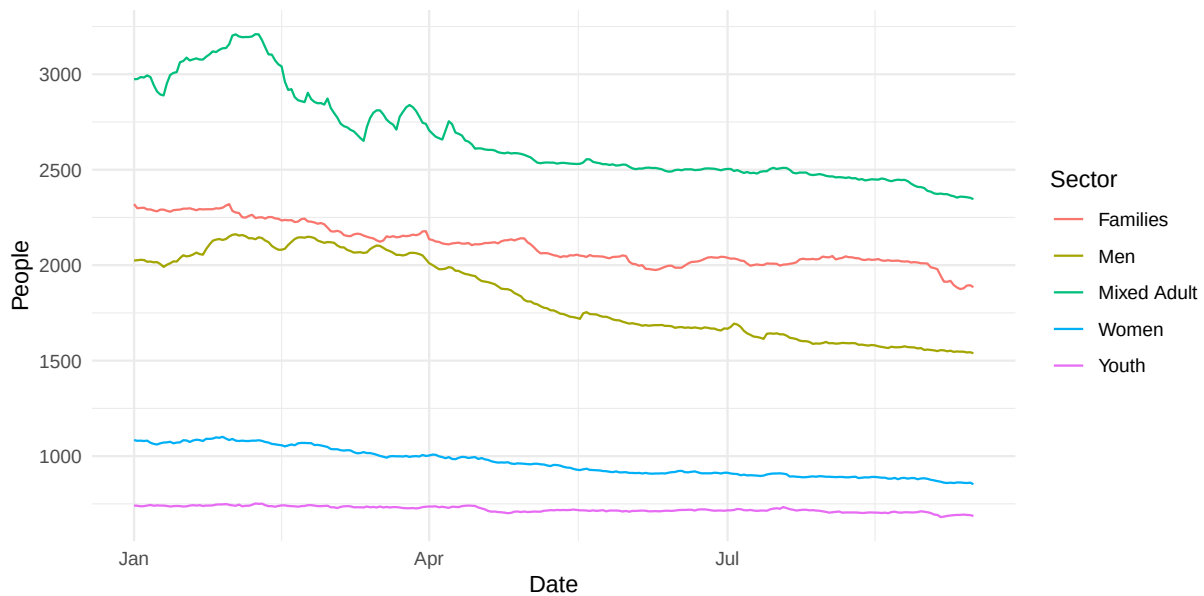


Figure 1: People in Toronto’s shelter system each night in 2026, by sector. Data from Open Data Toronto.

Figure 1 is a cross-reference: the `label` on the chunk starts with `fig-`, so Quarto numbers the figure and the reference updates itself if figures move. Three things: the graph has labelled axes with units, the legend is named, and the ordering of the lines carries information.

Step 4: a table

```
latest <-
  daily |>
  filter(date == max(date)) |>
  mutate(
    beds = if_else(capacity_beds > 0, occupied_beds / capacity_beds, NA),
    rooms = if_else(capacity_rooms > 0, occupied_rooms / capacity_rooms, NA)
  ) |>
  select(
    Sector = sector, People = service_users,
    `Bed occupancy` = beds, `Room occupancy` = rooms
  )

tt(latest, digits = 3) |>
  format_tt(
```

Table 1: Shelter occupancy on the most recent night in the data, by sector.

Sector	People	Bed occupancy	Room occupancy
Families	1885	82.4%	98.1%
Men	1539	95.3%	100.0%
Mixed Adult	2346	99.3%	100.0%
Women	853	96.8%	100.0%
Youth	687	93.3%	NA

```
j = c("Bed occupancy", "Room occupancy"),
fn = scales::label_percent(accuracy = 0.1)
) |>
format_tt(escape = TRUE)
```

Table 1 is the same idea for a table. The table has a caption that says what the rows are and when the data are from. In terms of measurement, notice that Families may be room-based and so could undercount.

Step 5: Citations

Citations come from `references.bib` and the `bibliography` line in the header. Writing `@healy2019` gives Healy (2026), and `[@healy2019]` gives a parenthetical (Healy 2026). The reference list is added at the end automatically.

Rendering

From the terminal, in the folder that has the file:

```
quarto render 02-demo.qmd
```

`format: pdf` in the header makes a PDF (this needs a LaTeX installation; `quarto install tinytex` gets one). Change it to `format: html` for a web page or `format: revealjs` for slides; nothing else in the file changes. In VS Code, the Quarto extension gives you a Render button and a preview.

Websites

A Quarto website is a folder with a `_quarto.yml` and some `.qmd` pages. `website_example/` in this week's folder is a working one; copy it. The three files that matter:

`_quarto.yml`:

```
project:
  type: website
  output-dir: docs

website:
  title: "Toronto shelters"
  navbar:
    left:
      - href: index.qmd
        text: Home
      - href: shelter.qmd
        text: Occupancy

format:
  html:
    theme: cosmo
```

`index.qmd` is the home page, `shelter.qmd` is a page with the graph and table above. Rendering the whole site:

```
quarto render
```

puts the finished HTML in `docs/`. That folder name is the trick for GitHub Pages: push the repo to GitHub, then under Settings, Pages, choose “Deploy from a branch”, branch `main`, folder `/docs`. A minute later the site is at <https://<username>.github.io/<repo>/>. Every time you render and push, the site updates. Commit `docs/`; it is the product.

Working in groups: pull requests and conflicts

Next week's ISLR/P group needs to get its slides into the class repo, and the Part 2 teams will share one repo for a month. The mechanism for both is the pull request.

The loop, for a repo you do not own:

```
# once: fork on GitHub, then
git clone https://github.com/<you>/<repo>.git
cd <repo>
git checkout -b week_03_group_1      # a branch for your change
# ... add your files ...
git add .
git commit -m "Add group 1 slides on ISLR 2.1"
git push -u origin week_03_group_1
```

Then on GitHub, “Compare & pull request”. The PR is where the review happens: someone else reads the diff, leaves comments on lines, and approves or asks for changes. Only after that does it get merged. Nobody pushes to `main` directly.

A conflict happens when two people change the same lines and the second one to merge has to choose. Git marks the file like this:

```
<<<<<<< HEAD
- Group 1: Chapter 2.1
=====
- Group 1: Chapter 5.1
>>>>>>> week_03_group_1
```

Open the file, keep the version you want, delete the three marker lines, then `git add` and `git commit`. That is all a conflict is. The Timbers reading has the longer version; the live demonstration will make one on purpose with two branches that edit the same line of a README.

Two rules that prevent most conflicts: pull before you start (`git pull`), and keep PRs small and about one thing.

What to do in the worksheet

Question 3 builds a website like `website_example/` and puts it on GitHub Pages. Question 2 is your first PR to the class repo. Question 4 is three graphs and a table of your own, on the website. If you finish, make a conflict with your neighbour and fix it.

References

Healy, Kieran. 2026. *Data Visualization: A Practical Introduction*. 2nd ed. Princeton University Press.